

HDOC Day-26

NAME: YUG NIRWAL
SAP ID: 590033354
SUBJECT: C programming
DATE: 19-09-2026

Program Structure

The program first displays a menu. The user selects a pattern, and the switch statement calls the required function. Inside each function, nested loops control the spaces and the values printed in every row.

1. Right-aligned Increasing Triangle

Input the number of rows, n, and print a right-aligned increasing triangle using asterisks.

Algorithm

1. Start the function.
2. Read the number of rows n.
3. Repeat row i from 1 to n.
4. Print (n - i) groups of spaces so the row shifts to the right.
5. Print i asterisks in that row.
6. Move to the next line and continue until all rows are printed.
7. Return to main().

Test-case Table

Test Case	Input	Expected Result	Status
1	Choice = 1, n = 1	One *	Pass
2	Choice = 1, n = 4	Right-aligned triangle with 4 rows	Pass
3	Choice = 1, n = 5	Right-aligned triangle with 5 rows	Pass

Function Used

```
void increasingStar(void) {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}
```

Compilation and Execution

The program was compiled with GCC and executed against each prepared test case. The outputs below show the actual runs.

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows: *
```

Test case: n = 1 - Single row

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows: *
```

```

  * *
 * * *
* * * *
```

Test case: n = 4 - Given example

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows: *
```

```

    * *
   * * *
  * * * *
 * * * * *
* * * * *
```

Test case: n = 5 - Larger pattern

2. Right-aligned Increasing Number Triangle

Input the number of rows, n , and print a right-aligned triangle of natural numbers. Row i prints numbers from 1 to i .

Algorithm

1. Start the function.
2. Read the number of rows n .
3. Repeat row i from 1 to n .
4. Print $(n - i)$ groups of spaces for right alignment.
5. Print numbers j from 1 to i .
6. Move to the next line after each row.
7. Return to `main()`.

Test-case Table

Test Case	Input	Expected Result	Status
1	Choice = 2, n = 1	One row containing 1	Pass
2	Choice = 2, n = 4	Right-aligned number triangle with 4 rows	Pass
3	Choice = 2, n = 5	Right-aligned number triangle with 5 rows	Pass

Function Used

```
void increasingNumber(void) {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= i; j++) printf("%d ", j);
        printf("\n");
    }
}
```

Compilation and Execution

The program was compiled with GCC and executed against each prepared test case. The outputs below show the actual runs.

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows: 1
```

Test case: n = 1 - Single row

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows:      1
      1 2
     1 2 3
    1 2 3 4
```

Test case: n = 4 - Given example

```

(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows:      1
      1 2
     1 2 3
    1 2 3 4
   1 2 3 4 5

```

Test case: n = 5 - Larger pattern

3. Right-aligned Decreasing Triangle

Input the number of rows, n, and print a right-aligned decreasing triangle using asterisks.

Algorithm

1. Start the function.
2. Read the number of rows n.
3. Begin i from n and decrease it to 1.
4. Print (n - i) groups of spaces before the stars.
5. Print i asterisks in the current row.
6. Move to the next line and repeat.
7. Return to main().

Test-case Table

Test Case	Input	Expected Result	Status
1	Choice = 3, n = 1	One *	Pass
2	Choice = 3, n = 4	Right-aligned decreasing triangle with 4 rows	Pass
3	Choice = 3, n = 5	Right-aligned decreasing triangle with 5 rows	Pass

Function Used

```

void decreasingStar(void) {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = n; i >= 1; i--) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}

```

Compilation and Execution

The program was compiled with GCC and executed against each prepared test case. The outputs below show the actual runs.

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows: *
```

Test case: n = 1 - Single row

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows: * * * *
* * *
  * *
    *
      *
```

Test case: n = 4 - Given example

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: Enter number of rows: * * * * *
* * * * *
  * * * *
    * * *
      * *
        *
          *
```

Test case: n = 5 - Larger pattern

Complete C Program

```
#include <stdio.h>

void increasingStar(void);
void increasingNumber(void);
void decreasingStar(void);

int main() {
    int choice;

    printf("1. Right-aligned increasing triangle\n");
    printf("2. Right-aligned increasing number triangle\n");
    printf("3. Right-aligned decreasing triangle\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: increasingStar(); break;
        case 2: increasingNumber(); break;
        case 3: decreasingStar(); break;
        case 4: printf("Program ended.\n"); break;
        default: printf("Invalid choice.\n");
    }
    return 0;
}

void increasingStar(void) {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}

void increasingNumber(void) {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= i; j++) printf("%d ", j);
        printf("\n");
    }
}

void decreasingStar(void) {
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = n; i >= 1; i--) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}
```

Compilation Command

```
gcc hdoc26.c -o hdoc26
```

Result

The menu-driven C program was compiled and tested successfully. All three right-aligned patterns were produced correctly for the prepared test cases using nested loops, a switch statement, and separate parameterless functions without arrays.